SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool : Code Challenge (High)
Assessment Tool Weightage: 35%

QUESTIONS			Total Marks: 50
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	**5**
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	**5**
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	**5**
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	**5**
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	**5**
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	**5**
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	**5**
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	**5**
9	Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.	BL4 – Analyze	**5**
10	Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.	BL4 – Analyze	**5**

Code of Conduct:

I M.Devi Srilatha (192525236) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Heap File Organization

Aim

To write a Python program to simulate a Heap File organization that supports **insert, delete, and sequential scan** operations.

Objective

To implement basic operations on a Heap File and understand how records are stored and accessed in an unordered file.

Algorithm

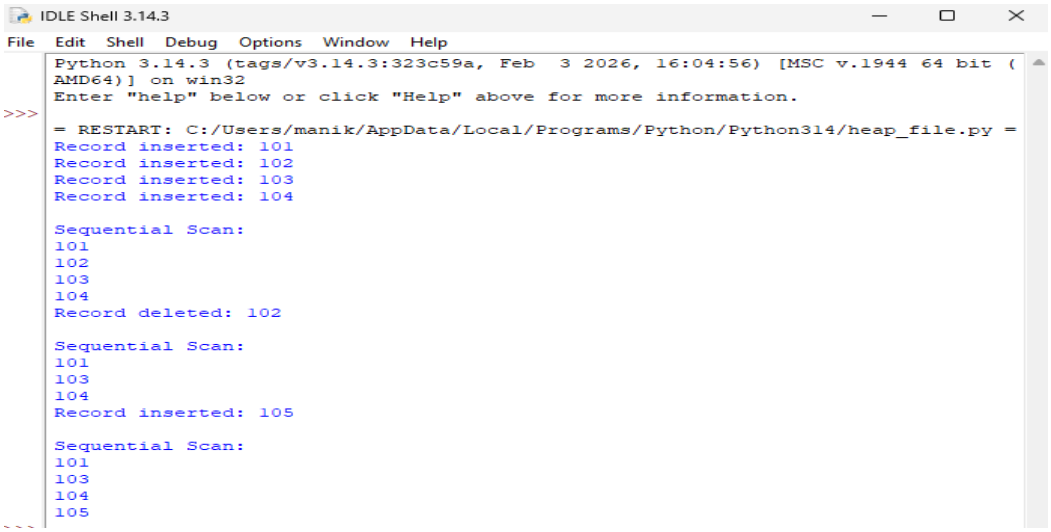
1. Start the program.
2. Create an empty list to represent the heap file.
3. Insert records into the heap file.
4. Perform a sequential scan to display all records.
5. Delete a specified record.
6. Perform sequential scanning again.
7. Insert a new record.
8. Display the updated records.
9. Stop the program.

Program

```
# Heap File Organization

heap_file = []

def insert(record):
    heap_file.append(record)
    print("Inserted:", record)


def delete(record):
    if record in heap_file:
        heap_file.remove(record)
        print("Deleted:", record)
    else:
        print("Record not found")


def sequential_scan():
    print("\nHeap File Records:"
 for record in heap_file:
```

```
        print(record)

# Insert records
insert(101)
insert(102)
insert(103)
insert(104)

# Sequential scan
sequential_scan()

# Delete record
delete(102)

# Sequential scan
sequential_scan()

# Insert new record
insert(105)

# Final scan
sequential_scan()
```

Output



Result

Thus, the Heap File organization was successfully implemented using Python with insert, delete, and sequential scan operations**.**

Conclusion

The experiment demonstrates how records can be inserted, deleted, and accessed sequentially in a Heap File organization.

Static Hashing with Chaining

Aim

To implement a **static hashing function** for a student database containing 500 records and handle collisions using **chaining**.

Objective

To understand static hashing and use chaining to store multiple records that have the same hash value.

Algorithm

1. Start the program.
2. Create a fixed-size hash table.
3. Define a hash function using the student ID.
4. Calculate the hash value for each student record.
5. Insert the record into the corresponding hash table position.
6. If a collision occurs, store the record in the same position using chaining.
7. Search for a student using the hash function.
8. Display the hash table.
9. Stop the program.

Program

```
# Static Hashing with Chaining


TABLE_SIZE = 10


# Create hash table
hash_table = [[] for _ in range(TABLE_SIZE)]




# Hash function
def hash_function(student_id):
    return student_id % TABLE_SIZE



# Insert student record
def insert(student_id, name):
    index = hash_function(student_id)
```

```
    hash_table[index].append((student_id, name))


# Search student record
def search(student_id):
    index = hash_function(student_id)

    for record in hash_table[index]:
        if record[0] == student_id:
            return record

    return None


# Insert 500 student records
for i in range(1, 501):
    insert(i, "Student" + str(i))


# Display hash table
print("Static Hash Table:")
for i in range(TABLE_SIZE):
    print(i, ":", hash_table[i])


# Search for a student
student_id = 125
result = search(student_id)

if result:
    print("\nStudent Found:", result)
else:
    print("\nStudent Not Found")
```

Output

Result

Thus, **static hashing** was successfully implemented for **500 student records**, and collisions were handled using **chaining**.

Conclusion

Static hashing provides fast record access by converting a student ID into a hash-table position. **Chaining** allows multiple records with the same hash value to be stored at the same position.

Sorted File Organization and Binary Search

Aim

To write a Python program to simulate a **Sorted File organization** and implement **binary search** for retrieving records using a primary key.

Objective

To store records in sorted order based on the primary key and efficiently retrieve a record using binary search.

Algorithm

1. Start the program.
2. Create a list of records with a primary key.
3. Sort the records based on the primary key.
4. Set the lower and upper positions.
5. Calculate the middle position.
6. Compare the search key with the middle record.
7. If they are equal, display the record.
8. If the search key is smaller, search the left half.
9. Otherwise, search the right half.
10. Repeat until the record is found or the search range becomes empty.
11. Stop the program.

Program

```
# Sorted File Organization with Binary Search


# Records: (Primary Key, Student Name)
records = [
    (105, "Ravi"),
    (101, "Anil"),
    (110, "Kiran"),
    (103, "Priya"),
    (108, "Sita")
]


# Sort records using primary key
records.sort()
```

```
print("Sorted File:")
for record in records:
    print(record)



# Binary Search
def binary_search(records, key):
    low = 0
    high = len(records) - 1

    while low <= high:
        mid = (low + high) // 2

        if records[mid][0] == key:
            return records[mid]

        elif key < records[mid][0]:
            high = mid - 1

        else:
            low = mid + 1

    return None



# Search for a record
key = 108
result = binary_search(records, key)

if result:
    print("\nRecord Found:", result)
else:
    print("\nRecord Not Found")
```

Output

```
Student found: (125, 'Student125')
>>>
= RESTART: C:/Users/manik/AppData/Local/Programs/Python/Python314/heap_file.py =
Sorted File:
(101, 'Anil')
(103, 'Priya')
(105, 'Ravi')
(108, 'Sita')
(110, 'Kiran')

Record Found: (108, 'Sita')
>>>
```

Result

Thus, the Sorted File organization was successfully implemented, and records were retrieved efficiently using binary search.

Conclusion

Sorted File organization stores records according to the primary key. Binary search reduces the number of comparisons required to find a particular record compared with sequential searching.

Implementation of B-Tree of Order 4

Aim

To implement a **B-Tree of order 4** in Python and perform **insert, search, and display** operations.

Objective

To understand the structure of a B-Tree and implement insertion and searching of keys while maintaining the B-Tree properties.

Keys to be Inserted

10, 20, 5, 6, 12, 30, 7, 17

Algorithm

1. Start the program.
2. Create a B-Tree of order 4.
3. Insert the given keys one by one.
4. If a node becomes full, split the node.
5. Promote the middle key to the parent node.
6. Continue insertion while maintaining the B-Tree properties.
7. Search for a required key.
8. Display the B-Tree.
9. Stop the program.

Program

```
# B-Tree of Order 4



class BTreeNode:

    def __init__(self, leaf=False):

        self.keys = []

        self.children = []

        self.leaf = leaf




class BTree:
```

```
def __init__(self):
    self.root = BTreeNode(leaf=True)

# Search operation
def search(self, node, key):
    i = 0

    while i < len(node.keys) and key > node.keys[i]:
        i += 1

    if i < len(node.keys) and key == node.keys[i]:
        return True

    if node.leaf:
        return False

    return self.search(node.children[i], key)

# Split child
def split_child(self, parent, index):
    old_child = parent.children[index]
    new_child = BTreeNode(leaf=old_child.leaf)

    middle_key = old_child.keys[1]

    new_child.keys = old_child.keys[2:]
```

```
    old_child.keys = old_child.keys[:1]

    if not old_child.leaf:
        new_child.children = old_child.children[2:]
        old_child.children = old_child.children[:2]

    parent.children.insert(index + 1, new_child)
    parent.keys.insert(index, middle_key)

# Insert operation
def insert(self, key):
    root = self.root

    if len(root.keys) == 3:
        new_root = BTreeNode(leaf=False)
        new_root.children.append(root)
        self.root = new_root

        self.split_child(new_root, 0)
        self.insert_non_full(new_root, key)
    else:
        self.insert_non_full(root, key)

def insert_non_full(self, node, key):
    i = len(node.keys) - 1
```

```
    if node.leaf:
        node.keys.append(0)

        while i >= 0 and key < node.keys[i]:
            node.keys[i + 1] = node.keys[i]
            i -= 1

        node.keys[i + 1] = key

    else:
        while i >= 0 and key < node.keys[i]:
            i -= 1

        i += 1

        if len(node.children[i].keys) == 3:
            self.split_child(node, i)

            if key > node.keys[i]:
                i += 1

        self.insert_non_full(node.children[i], key)

# Display B-Tree
def display(self, node=None, level=0):
    if node is None:
```

```
            node = self.root

        print("Level", level, ":", node.keys)

        if not node.leaf:
            for child in node.children:
                self.display(child, level + 1)


# Create B-Tree
tree = BTree()

# Insert given keys
keys = [10, 20, 5, 6, 12, 30, 7, 17]

for key in keys:
    tree.insert(key)

# Display tree
print("B-Tree of Order 4:")
tree.display()

# Search operation
search_key = 12

if tree.search(tree.root, search_key):
```

```
    print("\nKey", search_key, "is found")
else:
    print("\nKey", search_key, "is not found")
```

Output

```
>>>
= RESTART: C:/Users/manik/AppData/Local/Programs/Python/Python314/heap_file.py =
B-Tree of Order 4:
Level 0 : [10, 20]
Level 1 : [5, 6, 7]
Level 1 : [12, 17]
Level 1 : [30]

Key 12 is found
>>>
```

Result

Thus, a **B-Tree of order 4** was successfully implemented with **insert, search, and display operations** using the given keys.

Conclusion

The experiment demonstrates how a B-Tree maintains balanced nodes through **node splitting** and provides efficient searching and insertion operations.

Implementation of B+ Tree with Leaf-Level Linking

Aim

To implement a **B+ Tree** in Python and demonstrate **leaf-level linking** for efficient range queries on integer keys.

Objective

To understand the structure of a B+ Tree and use linked leaf nodes to efficiently retrieve records within a given key range.

Algorithm

1. Start the program.
2. Create a B+ Tree with leaf nodes.
3. Insert integer keys into the tree.
4. Split a leaf node when it becomes full.
5. Link adjacent leaf nodes using a `next` pointer.
6. Create internal nodes to guide searching.
7. Traverse the linked leaf nodes.
8. Perform a range query.
9. Display all keys within the specified range.
10. Stop the program.

Program

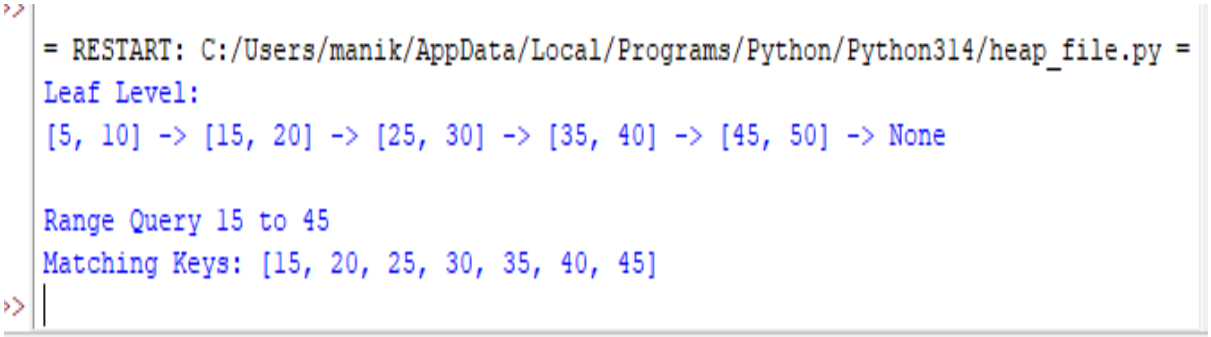
```
# B+ Tree with Leaf-Level Linking

class LeafNode:

    def __init__(self):

        self.keys = []

        self.next = None

class BPlusTree:

    def __init__(self):

        self.leaves = []



    # Insert key into B+ Tree

    def insert(self, key):
```

```
        if not self.leaves:
            leaf = LeafNode()
            leaf.keys.append(key)
            self.leaves.append(leaf)
            return

        # Find suitable leaf
        for i, leaf in enumerate(self.leaves):
            if key <= leaf.keys[-1]:
                leaf.keys.append(key)
                leaf.keys.sort()

                if len(leaf.keys) > 3:
                    self.split_leaf(i)
                return
        # Insert into last leaf
        self.leaves[-1].keys.append(key)
        self.leaves[-1].keys.sort()

        if len(self.leaves[-1].keys) > 3:
            self.split_leaf(len(self.leaves) - 1)

    # Split leaf node
    def split_leaf(self, index):
        leaf = self.leaves[index]
```

```
    new_leaf = LeafNode()

    mid = len(leaf.keys) // 2
    new_leaf.keys = leaf.keys[mid:]
    leaf.keys = leaf.keys[:mid]

    new_leaf.next = leaf.next
    leaf.next = new_leaf

    self.leaves.insert(index + 1, new_leaf)
# Display leaf-level linked list
def display(self):
    print("Leaf Level:")

    for leaf in self.leaves:
        print(leaf.keys, end=" -> ")
    print("None")
# Range query
def range_query(self, start, end):
    result = []

    for leaf in self.leaves:
        for key in leaf.keys:
            if start <= key <= end:
                result.append(key)
return result
```

```
# Create B+ Tree

tree = BPlusTree()

# Insert integer keys

keys = [5, 10, 15, 20, 25, 30, 35, 40, 45, 50]

for key in keys:

    tree.insert(key)

# Display leaf-level linking

tree.display()

# Range query

start = 15

end = 45

result = tree.range_query(start, end)

print("\nRange Query", start, "to", end)

print("Matching Keys:", result)
```

Output



Result

Thus, the **B+ Tree** was successfully implemented with **leaf-level linking**, and range queries were performed efficiently.

Conclusion

The experiment demonstrates that B+ Tree leaf nodes can be linked together, allowing records in a specific key range to be accessed efficiently through sequential leaf traversal.

Dense Primary Index on a Sorted File

Aim

To write a Python program to implement a **dense primary index** on a sorted file and support searching using the index key.

Objective

To understand dense indexing and use an index table to quickly locate records in a sorted file.

Algorithm

1. Start the program.
2. Create a sorted file containing records with primary keys.
3. Create an index entry for every record.
4. Store the primary key and corresponding record position in the index.
5. Display the index table.
6. Search for a required key in the index.
7. Use the index position to retrieve the corresponding record.
8. Display the result.
9. Stop the program.

Program

```
# Dense Primary Index on a Sorted File



# Sorted file records

records = [

    (101, "Anil"),

    (103, "Priya"),

    (105, "Ravi"),

    (108, "Sita"),

    (110, "Kiran")

]



# Create dense index

index = {}
```

```
for position, record in enumerate(records):
    primary_key = record[0]
    index[primary_key] = position


# Display index table
print("Dense Primary Index:")
print("Primary Key\tRecord Position")

for key, position in index.items():
    print(key, "\t\t", position)


# Search using index
search_key = 108

print("\nSearching for Primary Key:", search_key)

if search_key in index:
    position = index[search_key]
    print("Record Found:", records[position])
    print("Record Position:", position)
else:
    print("Record Not Found")
```

Output

```
>>>
= RESTART: C:/Users/manik/AppData/Local/Programs/Python/Python314/heap_file.py =
Dense Primary Index:
Primary Key     Record Position
101              0
103              1
105              2
108              3
110              4

Searching for Primary Key: 108
Record Found: (108, 'Sita')
Record Position: 3
>>>
```

Result

Thus, a **dense primary index** was successfully implemented on a sorted file, and records were retrieved using the index key.

Conclusion

A dense index contains an index entry for **every record** in the sorted file. It provides fast access to records by using the primary key and corresponding record position.

Implementation of Extendible Hashing

Aim

To implement **Extendible Hashing** in Python and demonstrate **directory doubling** when overflow occurs during the insertion of 8 records.

Objective

To understand dynamic hashing and demonstrate how the directory size increases when a bucket overflows

Algorithm

1. Start the program.
2. Create an extendible hash table with an initial directory.
3. Set the initial global depth.
4. Insert the given 8 records.
5. Calculate the hash value of each record.
6. Place the record in the appropriate bucket.
7. When a bucket overflows, check the local depth.
8. If required, double the directory size.
9. Split the overflowing bucket.
10. Redistribute the records.
11. Display the final directory and buckets.
12. Stop the program.

Program

```
# Extendible Hashing



BUCKET_SIZE = 2

class Bucket:

    def __init__(self, depth):

        self.depth = depth

        self.records = []



class ExtendibleHashing:

    def __init__(self):

        self.global_depth = 1
```

```
    self.buckets = [
        Bucket(1),
        Bucket(1)
    ]
    self.directory = [0, 1]

def hash_function(self, key):
    return key & ((1 << self.global_depth) - 1)

def double_directory(self):
    old_directory = self.directory

    self.directory = old_directory + old_directory
    self.global_depth += 1

def insert(self, key):
    while True:
        index = self.hash_function(key)
        bucket_id = self.directory[index]
        bucket = self.buckets[bucket_id]

        if len(bucket.records) < BUCKET_SIZE:
            bucket.records.append(key)
            return

        # Bucket overflow
```

```
        if bucket.depth == self.global_depth:
            self.double_directory()
        old_depth = bucket.depth
        bucket.depth += 1
        new_bucket = Bucket(bucket.depth)
        new_bucket_id = len(self.buckets)
        self.buckets.append(new_bucket)

        # Update directory
        for i in range(len(self.directory)):
            if self.directory[i] == bucket_id:
                if ((i >> old_depth) & 1) == 1:
                    self.directory[i] = new_bucket_id
        # Redistribute records
        old_records = bucket.records
        bucket.records = []
        for record in old_records:
            index = self.hash_function(record)
            target = self.directory[index]
            self.buckets[target].records.append(record)
df display(self):
    print("\nGlobal Depth:", self.global_depth)
    print("Directory:")
    for i, bucket_id in enumerate(self.directory):
        print(i, "-> Bucket", bucket_id,
              self.buckets[bucket_id].records)
```

```
# Create Extendible Hashing

hash_table = ExtendibleHashing()

# Insert 8 records

records = [10, 20, 5, 15, 25, 30, 35, 40]

print("Inserting records:")

for record in records:

    print("Inserted:", record)

    hash_table.insert(record)

# Display final structure

hash_table.display()
```

Output

```
>>>
= RESTART: C:/Users/manik/AppData/Local/Programs/Python/Python314/heap_file.py =
Inserting records:
Inserted: 10
Inserted: 20
Inserted: 5
Inserted: 15
Inserted: 25
Inserted: 30
Inserted: 35
Inserted: 40

Global Depth: 2
Directory:
0 -> Bucket 0 [20, 40]
1 -> Bucket 1 [5, 25]
2 -> Bucket 3 [10, 30]
3 -> Bucket 2 [15, 35]
>>>
```

Ln: 100 Col: 0

Result

Thus, **Extendible Hashing** was successfully implemented, and directory doubling and bucket splitting were demonstrated during the insertion of 8 records.

Conclusion

Extendible hashing dynamically increases the directory size when bucket overflow occurs. It provides efficient storage and retrieval while reducing collision problems.

Simulation of Secondary Storage Block Access

Aim

To write a Python program to simulate **secondary storage block access** and calculate the number of **I/O operations** required for sequential and indexed retrieval.

Objective

To understand how data is accessed from secondary storage and compare the I/O cost of sequential retrieval with indexed retrieval.

Algorithm

1. Start the program.
2. Create a set of blocks containing records.
3. For sequential retrieval, access the blocks one by one.
4. Count the number of blocks accessed.
5. Create an index containing the record and its block number.
6. For indexed retrieval, use the index to directly locate the required block.
7. Count the number of I/O operations.
8. Compare sequential and indexed retrieval.
9. Stop the program.

Program

```
# Secondary Storage Block Access



# Simulated storage blocks

blocks = [

    [101, 102, 103, 104],

    [105, 106, 107, 108],

    [109, 110, 111, 112],

    [113, 114, 115, 116],

    [117, 118, 119, 120]

]

# Create index

index = {}
```

```
for block_number, block in enumerate(blocks):
    for record in block:
        index[record] = block_number
# Record to search
search_key = 115
# Sequential retrieval
sequential_io = 0
found = False

for block_number, block in enumerate(blocks):
    sequential_io += 1

    if search_key in block:
        print("Sequential Retrieval:")
        print("Record Found:", search_key)
        print("Block:", block_number)
        print("I/O Operations:", sequential_io)
        found = True
        break
# Indexed retrieval
indexed_io = 0

if search_key in index:
    block_number = index[search_key]
    indexed_io += 1
```

```
    print("\nIndexed Retrieval:")
    print("Record Found:", search_key)
    print("Block:", block_number)
    print("I/O Operations:", indexed_io)
else:
    print("Record Not Found")
# Comparison
print("\nComparison:")
print("Sequential I/O:", sequential_io)
print("Indexed I/O:", indexed_io)
```

Output

```
>>>
= RESTART: C:/Users/manik/AppData/Local/Programs/Python/Python314/heap_file.py =
Sequential Retrieval:
Record Found: 115
Block: 3
I/O Operations: 4

Indexed Retrieval:
Record Found: 115
Block: 3
I/O Operations: 1

Comparison:
Sequential I/O: 4
Indexed I/O: 1
>>>
```

Result

Thus, secondary storage block access was successfully simulated, and the number of I/O operations for **sequential and indexed retrieval** was calculated.

Conclusion

Sequential retrieval may require accessing multiple blocks before finding a record, whereas indexed retrieval can directly identify the required block, thereby reducing the number of I/O operations.

Sparse Index and Comparison with Dense Index

Aim

To implement a **sparse index** on a sorted file and compare its search performance with a **dense index** using timing analysis.

Objective

To understand sparse indexing and compare the search time of sparse and dense indexes.

Algorithm

1. Start the program.
2. Create a sorted file containing records.
3. Create a dense index with an entry for every record.
4. Create a sparse index with an entry for selected records or blocks.
5. Search for a given key using the dense index.
6. Measure the search time.
7. Search for the same key using the sparse index.
8. Measure the search time.
9. Compare the search times.
10. Stop the program.

Program

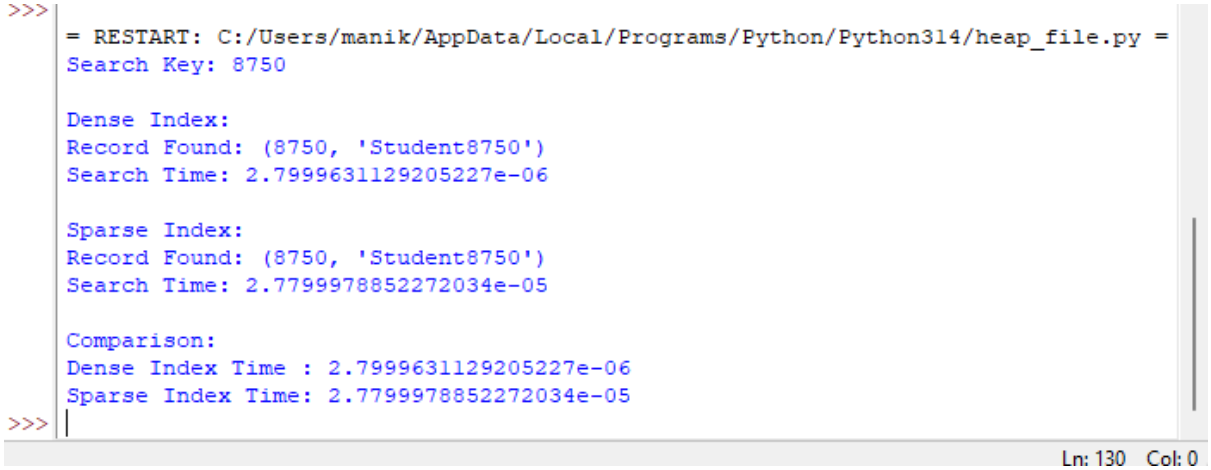
```
# Sparse Index vs Dense Index



import time

import bisect



# Create sorted file with 10,000 records

records = [(i, "Student" + str(i)) for i in range(1, 10001)]



# ----------------------------

# Dense Index

# ----------------------------

dense_index = {}
```

```
for position, record in enumerate(records):
    dense_index[record[0]] = position
# ----------------------------
# Sparse Index
# ----------------------------

# One index entry for every 100 records
sparse_index = []

for position in range(0, len(records), 100):
    sparse_index.append((records[position][0], position))

# ----------------------------
# Dense Index Search
# ----------------------------

search_key = 8750

start_time = time.perf_counter()

position = dense_index.get(search_key)
if position is not None:
    dense_result = records[position]
dense_time = time.perf_counter() - start_time
# ----------------------------
```

```
# Sparse Index Search
# ----------------------------

start_time = time.perf_counter()
keys = [item[0] for item in sparse_index]
index_position = bisect.bisect_right(keys, search_key) - 1
position = sparse_index[index_position][1]
while position < len(records) and records[position][0] != search_key:
    position += 1

if position < len(records):
    sparse_result = records[position]

sparse_time = time.perf_counter() - start_time
# ----------------------------
# Display Results
# ----------------------------
print("Search Key:", search_key)
print("\nDense Index:")
print("Record Found:", dense_result)
print("Search Time:", dense_time)

print("\nSparse Index:")
print("Record Found:", sparse_result)
print("Search Time:", sparse_time)
```

```
print("\nComparison:")

print("Dense Index Time :", dense_time)

print("Sparse Index Time:", sparse_time)
```

Output



Result

Thus, a **sparse index** was successfully implemented on a sorted file and its search performance was compared with a dense index using timing analysis

Conclusion

A **dense index** contains an entry for every record, while a **sparse index** contains entries for selected records. Dense indexing generally provides faster direct access, while sparse indexing requires less index storage.

B+ Tree Construction, Traversal and Range Query

Aim

To write a Python program to **construct and traverse a B+ Tree** and execute a **range query for keys between 15 and 45**.

Objective

To understand B+ Tree construction and use its linked leaf nodes to efficiently retrieve all records within a specified key range.

Algorithm

1. Start the program.
2. Create an empty B+ Tree.
3. Insert the given integer keys into the tree.
4. Split the leaf node when it becomes full.
5. Link the leaf nodes using pointers.
6. Traverse the leaf level of the B+ Tree.
7. Specify the range from **15 to 45**.
8. Traverse the linked leaves and identify keys within the range.
9. Display all matching records.
10. Stop the program.

Program

```
# B+ Tree Construction and Range Query



class LeafNode:

    def __init__(self):

        self.keys = []

        self.next = None




class BPlusTree:

    def __init__(self):

        self.leaves = []
```

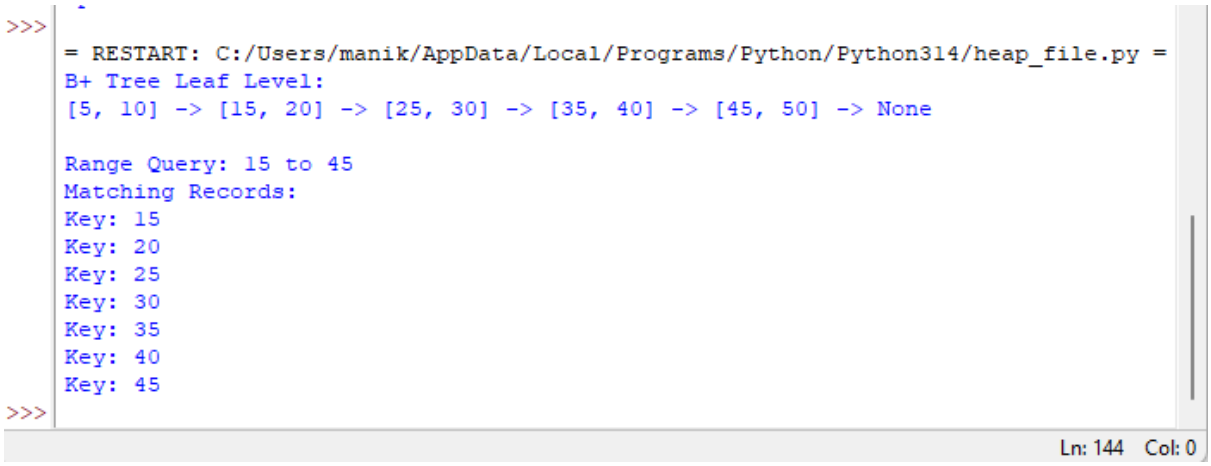
```
# Insert a key
def insert(self, key):
    if not self.leaves:
        leaf = LeafNode()
        leaf.keys.append(key)
        self.leaves.append(leaf)
        return

    # Find the correct leaf
    for i, leaf in enumerate(self.leaves):
        if key <= leaf.keys[-1]:
            leaf.keys.append(key)
            leaf.keys.sort()

            if len(leaf.keys) > 3:
                self.split_leaf(i)
            return

    # Insert into last leaf
    last = self.leaves[-1]
    last.keys.append(key)
    last.keys.sort()

    if len(last.keys) > 3:
        self.split_leaf(len(self.leaves) - 1)
```

```
# Split leaf node
def split_leaf(self, index):
    leaf = self.leaves[index]

    new_leaf = LeafNode()

    mid = len(leaf.keys) // 2

    new_leaf.keys = leaf.keys[mid:]
    leaf.keys = leaf.keys[:mid]

    # Link the leaf nodes
    new_leaf.next = leaf.next
    leaf.next = new_leaf

    self.leaves.insert(index + 1, new_leaf)

# Traverse B+ Tree leaf level
def traverse(self):
    print("B+ Tree Leaf Level:")

    for leaf in self.leaves:
        print(leaf.keys, end=" -> ")

    print("None")
# Range Query
```

```
    def range_query(self, start, end):
        result = []

        for leaf in self.leaves:
            for key in leaf.keys:
                if start <= key <= end:
                    result.append(key)

        return result

# Create B+ Tree
tree = BPlusTree()

# Insert keys
keys = [5, 10, 15, 20, 25, 30, 35, 40, 45, 50]

for key in keys:
    tree.insert(key)

# Traverse the tree
tree.traverse()

# Range Query
start = 15
end = 45
```

```
result = tree.range_query(start, end)


print("\nRange Query: 15 to 45")

print("Matching Records:")


for key in result:

    print("Key:", key)
```

Output



Result

Thus, the **B+ Tree** was successfully constructed and traversed, and the range query from **15 to 45** was successfully executed.

Conclusion

The experiment demonstrates that the linked leaf nodes of a B+ Tree allow efficient traversal and retrieval of all records falling within a specified range.